

Laboratorio Perceptron: digitos 0 a 9

Aplicacao desktop didatica em Python, com desenho de pixels, treinamento animado, classificacao animada e auditoria numerica dos pesos. Uma unica camada linear conecta 64 entradas a 10 neuronios de saida. Nao e uma MLP, nao possui camada oculta e nao usa backpropagation.

Instalacao e execucao

Requer Python 3.11 ou mais recente com Tkinter. Validado no Windows com Python 3.14.6. O instalador oficial do Python para Windows inclui Tcl/Tk; Tkinter nao deve ser instalado por pip.

No PowerShell, dentro desta pasta, usando o ambiente virtual da pasta pai:

```
& '..\.venv\Scripts\python.exe' -m pip install -r requirements.txt  
& '..\.venv\Scripts\python.exe' interface.py
```

Para um ambiente novo e independente, dentro desta pasta:

```
py -m venv .venv  
& '..\.venv\Scripts\python.exe' -m pip install -r requirements.txt  
& '..\.venv\Scripts\python.exe' interface.py
```

Use `pip install -r requirements.txt`, com `-r`. Invocar um executavel entre aspas no PowerShell exige o operador `&` antes do caminho. Usar `python.exe -m pip` garante a instalacao no mesmo interpretador da aplicacao. Nao ha servidor web, conta, chave de API ou download de dataset em tempo de execucao.

Experimento sugerido

Em **Entrada**, selecione a codificacao antes de iniciar o treino. O padrao e **Bipolar (-1/+1)**. O seletor fica bloqueado durante a classificacao e depois de iniciar o treino; **Reiniciar rede** libera a escolha e limpa pesos, metricas e referencias. Trocar a codificacao antes do treino tambem reinicia esses estados, preservando o desenho original.

1. Selecione **Detalhado** e pressione **Passo** tres vezes: entrada, somas antes da correcao e atualizacao. **Treinar** automatiza essas fases; **Pausar/Continuar** preserva a amostra corrente. Ajuste o intervalo em ms.
2. Em **Neuronio em foco**, escolha uma saida de 0 a 9. O grafo mostra todos os neuronios, mas apenas as 64 conexoes dessa saida para evitar sobreposicao de 640 arestas. Verde indica contribuicao positiva, rosa negativa, e a espessura indica magnitude relativa. O vencedor tem preenchimento verde; o rotulo esperado tem contorno rosa antes da atualizacao.
3. Use **Rapido (50 amostras/quadro)** para completar as 20 epocas. Todas as amostras continuam sendo treinadas e registradas; apenas os quadros visuais intermediarios sao omitidos. A selecao do modo pode mudar durante o treino.

4. Consulte **Treinamento** para acuracia ao final de cada epoca, numero de erros antes da atualizacao e matriz de confusao do teste com os pesos atuais.
5. Carregue uma amostra de **Teste**, ou desenhe na grade com o botao esquerdo. Arraste para pintar; use o botao direito ou **Borracha** para apagar. O controle de intensidade permite tons de cinza. **Limpar** zera a grade.
6. Pressione **Classificar**. A animacao revela as dez somas em sequencia e destaca a maior. A classificacao nao modifica pesos. Uma grade vazia e rejeitada; uma rede ainda nao treinada e identificada no estado da tela.
7. Abra **Auditoria dos pesos**, escolha o neuronio e confira a soma das contribuicoes e do bias. **Capturar pesos de referencia** fixa uma copia para comparar com atualizacoes posteriores. **Reiniciar rede** limpa pesos, historico e referencia e libera os parametros para um novo experimento.

O desenho permanece separado da amostra apresentada pelo treino no grafo. Durante uma amostra de treino, conclua suas fases com **Passo** antes de carregar ou classificar outra entrada. Codificacao, taxa e total de epocas ficam fixos ate reiniciar a rede. Os controles da interface sao executados por callbacks Tkinter, sem um loop de treinamento bloqueando a janela inteira.

Codificacao dos pixels

A grade, a galeria e a base original continuam armazenadas entre 0 e 1. Uma unica funcao, `encode_pixels`, converte treino, teste e desenho para a codificacao selecionada. O perceptron recebe vetores ja convertidos.

Modo	Intensidade abaixo de 0.5	Intensidade igual ou acima de 0.5
Bipolar (-1/+1), padrao	-1	+1
Binaria (0/1)	0	1
Tons de cinza (0 a 1)	Mantem intensidade original	Mantem intensidade original

O limiar e fixo em 0.5 nos dois modos discretos. Nao ha tons intermediarios no modo bipolar: a conversao e $2 * (\text{pixels} \geq 0.5) - 1$, e nao simplesmente $2 * \text{pixels} - 1$. O slider continua alterando a intensidade original do desenho; o grafo mostra a entrada efetiva e rotula cada pixel bipolar como -1 ou +1. Desenhos que viram somente fundo depois da conversao sao rejeitados como vazios.

Um pixel -1 contribui com o negativo do peso. Na atualizacao da classe correta, ele diminui o peso em vez de deixa-lo inalterado, como aconteceria com entrada 0. Os rotulos de classe continuam sendo os digitos 0 a 9, sem conversao bipolar.

Exemplo para usar o modelo sem a interface:

```
from dataset import encode_pixels, load_reference_dataset
from perceptron import MultiClassPerceptron

data = load_reference_dataset()
train = encode_pixels(data.train_images, mode='bipolar')
test = encode_pixels(data.test_images, mode='bipolar')
model = MultiClassPerceptron()
```

```
model.fit(list(zip(train, data.train_labels)), epochs=20)
print(model.evaluate(list(zip(test, data.test_labels))))
```

Regra de aprendizado

Para a classe k , entrada x , pesos w_k e bias b_k :

$$s_k = w_k^T x + b_k, \quad \hat{y} = \operatorname{argmax}_k s_k$$

Se a previsao $\hat{y}$ difere do rotulo correto y , com taxa η :

$$w_y \leftarrow w_y + \eta x, \quad b_y \leftarrow b_y + \eta$$

$$w_{\hat{y}} \leftarrow w_{\hat{y}} - \eta x, \quad b_{\hat{y}} \leftarrow b_{\hat{y}} - \eta$$

As outras saidas nao mudam; em caso de acerto, nenhum peso muda. Pesos e biases comecam em zero.

Empates usam a classe de menor indice, como no `numpy.argmax`. As pontuacoes sao **somas lineares, nao probabilidades**. A animacao e uma demonstracao sequencial da operacao matricial, nao uma simulacao biologica nem uma rede de disparos temporais.

Auditoria e reproducao

- A tabela mostra entrada, peso de referencia, peso atual, diferenca da referencia, peso antes do ultimo passo, diferenca desse passo e `x * peso`. As linhas sao ordenadas pela magnitude da contribuicao; o bias fica ao final.
- Mapas 8x8 comparam referencia e peso atual na mesma escala simetrica. O mapa de diferencas tem escala propria, indicada no titulo. Verde e positivo; rosa e negativo. Coordenadas da tabela comecam em zero, em ordem linha/coluna.
- **CSV** exporta 650 linhas: 64 pesos e um bias para cada uma das 10 classes, com contribuicoes, pontuacoes completas, codificacao e limiar. A soma das contribuicoes de uma classe, incluindo o bias, reproduz sua pontuacao.
- **JSON** exporta pesos, biases, entrada analisada, contribuicoes, pontuacoes, referencia, indices de treino/teste, descricao da base, metricas e o registro de cada amostra: epoca, indice original, rotulo, previsao anterior, pontuacoes, taxa e se houve atualizacao. Inclui `encoding` e `threshold`; `features` ja contem a entrada convertida, enquanto `normalization` descreve a divisao inicial dos pixels por 16. O registro, aplicado desde pesos zerados sobre a mesma base convertida com essa codificacao e limiar, reproduz os pesos finais. Nao converta `features` novamente para reconstruir as pontuacoes exportadas. A exportacao nao e um mecanismo de carregamento/retomada de modelo nesta versao.
- A referencia capturada e independente da copia do ultimo passo. Ambas ficam em memoria ate reiniciar ou fechar; exporte para preservar a auditoria.

Base de referencia e limites

`sklearn.datasets.load_digits` inclui 1.797 imagens reais manuscritas de 8x8 pixels da base **UCI Optical Recognition of Handwritten Digits**. Autores: E. Alpaydin e C. Kaynak. As intensidades originais variam de 0 a 16 e sao divididas por 16 para obter imagens entre 0 e 1 antes da codificacao.

- Documentacao: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html
- Fonte: <https://archive.ics.uci.edu/dataset/80/optical+recognition+of+handwritten+digits>

- Separacao estratificada fixa: 1.347 amostras de treino e 450 de teste, semente 42. As epocas embaralham apenas o treino, tambem com semente 42.
- O teste nunca atualiza pesos. Suas metricas sao exibidas para fins didaticos; se usadas para escolher parametros, deixam de ser uma estimativa independente final. Em um estudo formal, acrescente validacao e um teste final intocado.
- Verificacao local com 20 epocas e taxa 0.1: bipolar com 96,44% no treino e 88,67% no teste; binaria com 95,77% e 89,33%; tons de cinza com 96,07% e 91,56%. Os modos usam a mesma divisao e ordem de amostras. A codificacao bipolar nao garante maior acuracia, e discretizar elimina informacao de intensidade. Isso nao garante o mesmo desempenho em desenhos novos com o mouse.
- Centralize o digito e use tons de cinza semelhantes aos exemplos. Nao ha normalizacao automatica de posicao, rotacao ou espessura do traco. Uma rede linear nao separa todos os casos; o erro pode oscilar durante o treinamento.
- Entradas nao vazias sempre recebem uma classe, mesmo sem parecerem digitos. Nao ha detector de imagens fora da distribuicao nem confianca calibrada.

Arquivos e testes

- `dataset.py`: carregamento, normalizacao, codificacao e separacao da base.
- `perceptron.py`: modelo, um passo de treino, inferencia e inspecao numerica.
- `visualization.py`: desenho do grafo em Tkinter.
- `interface.py`: interface, animacoes, curvas, auditoria e exportacoes.
- `test_perceptron.py`: testes do modelo e dos fluxos reais da interface.
- `requirements.txt`: dependencias externas; unittest e Tkinter sao da biblioteca padrao.

```
& '..\.venv\Scripts\python.exe' -m unittest discover -s . -p test_perceptron.py -v
```

Os testes da interface precisam de uma sessao grafica e abrem janelas brevemente. Verificam desenho e borracha, fases do treino, classificacao sem alterar pesos, renderizacao dos graficos, CSV/JSON, independencia da referencia, equivalencia do modo rapido e reproducao do historico nas tres codificacoes. Tambem verificam o limiar, a entrada negativa, os metadados exportados e o bloqueio da troca de codificacao depois do treino. Para testar somente o nucleo:

```
& '..\.venv\Scripts\python.exe' -m unittest test_perceptron.PerceptronTests -v
```